7153CEM BIG DATA ANALYTICS AND DATA VISUALISATION

Predictive Crop yield in india Data Analysis Using PySpark and Tableau: A Comprehensive Approach

Student Name: Sravani Neelakantam
Student ID: 14554384
Student Course Name: Data Science
Student Mail ID: neelakants@coventry.ac.uk

Contents

S.No	Title	Pg.No:
1	Abstract	2
2	Introduction	3
3	Background and Related work	3-4
4	Dataset Description	4
5	Methodology	5
6	Experimental Setup	6
7	Result Discussion	8
8	Conclusion and Future works	8
9	Social Impact	9
	Appendix 1	12
	Appendix 2	17

1. Abstract

In this project, we used Tableau and PySpark to do a thorough data analysis on a crop yield dataset. The dataset was processed and analysed to provide insights about crop yields across different areas and seasons. The features included in the dataset were crop type, season, state, area, production, yearly rainfall, fertiliser usage, and pesticide usage.

Using PySpark, we first preprocessed the data to handle missing values, encode categorical variables, and put feature vectors together. To forecast crop yields, regression models such as Random Forest Regressor, Decision Tree Regressor, and Linear Regression were tested and trained. Furthermore, we implemented a binary classification task that employs models like Random Forest Classifier, Decision Tree Classifier, and Logistic Regression to forecast crop yields that fall within or above the median value.

The performance of each model was assessed using evaluation metrics, such as accuracy and AUC for classification models and RMSE and R2 for regression models. The findings provide light on the variables affecting agricultural yields and the efficacy of various prediction algorithms.

Tableau was used to create interactive visualisations that help understand the distribution and relationships between the data pieces. Thanks to these visualisations, stakeholders can easily comprehend the key findings of the investigation, including how different factors affect crop yields and how well various predictive models work.

Our investigation shows how many factors affect crop productivity and how machine learning techniques may be used in agriculture for predictive analytics. With PySpark for data processing and Tableau for visualisation, we were able to manage the dataset effectively and deliver the findings in an understandable way.

Keywords: PySpark, Tableau, Crop Yield, Regression Analysis, Classification, Data Preprocessing, Predictive Analytics, Machine Learning, Agriculture

2. Introduction:

Crop yield forecasting and knowledge are essential for contemporary agriculture's ability to provide food security, prepare for the future, and use resources effectively. Advanced data analytics approaches can be utilised to extract useful insights and improve decision-making processes, given the growing availability of agricultural data. In this project, Tableau will be used for data visualisation and PySpark for machine learning and data processing in order to analyse an extensive crop yield dataset.

Various crop production-related parameters, including crop type, season, state, harvested area, production quantities, annual rainfall, fertiliser and pesticide consumption, are included in the dataset utilised for this study. These characteristics offer a solid foundation for investigating the variables affecting crop yields in various locations and throughout various seasons.

We used a methodical strategy to preprocess the data, create predictive models, and visualise the outcomes in order to meet our goals. PySpark's powerful machine learning libraries and ability to handle big datasets made it our only choice for data preprocessing and model training. To be more precise, we classified yield levels and predicted crop yields using regression and classification tasks, respectively.

To forecast the crop production based on the input features, we trained a number of models for the regression analysis, including Linear Regression, Decision Tree Regressor, and Random Forest Regressor. To facilitate classification, we trained models such Random Forest Classifier, Decision Tree Classifier, and Logistic Regression, and introduced a binary target variable that indicates if the crop yield is above or below the median value.

Interactive visualisations that aid in comprehending the distribution and connections between the data elements were produced using Tableau. Stakeholders may quickly understand the main conclusions drawn from the analysis—such as how various factors affect crop yields and how well different predictive models perform—thanks to these visualisations.

This study shows how to improve agricultural planning and productivity by integrating big data analytics and visualisation tools. The results of this research can help allocate resources, shape legislation, and eventually lead to more sustainable farming methods.

Importance of the Project:

Accurately forecasting crop yields can aid farmers in streamlining their operations, cutting down on waste, and enhancing crop management in general. It can also help stakeholders and governments make educated choices about food supply chains, costs, and agricultural regulations. This research aims to improve prediction accuracy in agricultural contexts and provide important insights into the factors impacting crop productivity by utilising data-driven methodologies.

3.1 Background:

The significance of crop yield prediction for agricultural planning and food security has led to a great deal of research on the topic. While statistical models and historical data were frequently the foundation of traditional approaches, current developments in data analytics and machine learning have created new opportunities for more precise and dynamic forecasts. When big data technologies, like PySpark, are integrated, huge datasets may be processed and analysed, revealing intricate patterns and relationships that were previously hard to find.

3.2 Related Work:
The use of machine learning models for crop yield prediction has been investigated in a number of research. Neural networks, support vector machines, decision trees, and regression models are examples of common methodologies. For example:

Regression Models: Because of their ease of use and interpretability, linear and multiple regression models are frequently employed. Based on a linear connection between the dependent variable (yield) and one or more independent variables (features), these models forecast crop yields.
Models like decision trees and random forests perform better in predictions because they can capture non-linear correlations and interactions between features. Research have demonstrated that yield forecast accuracy can be greatly increased by using ensemble techniques such as Random Forests.
Neural Networks: Complex, non-linear correlations in agricultural data have been modelled using deep learning methods, in particular neural networks. Although these models can produce extremely accurate predictions, they do require enormous datasets and significant computational resources.
Big Data with PySpark: Processing large-scale agricultural datasets efficiently is made possible by the use of big data frameworks like PySpark. PySpark is a perfect tool for this kind of analysis because it can manage machine learning pipelines and distributed data processing.

4. Dataset Description:

The dataset for this project comprises a diverse set of attributes related to crop production. Specifically, the dataset includes the following columns:

- **Crop**: Type of crop.
- **Crop_Year**: Year of cultivation.
- **Season**: Season during which the crop was cultivated.
- **State**: State where the crop was cultivated.
- **Area**: Area harvested (in hectares).
- **Production**: Production quantity (in tonnes).
- **Annual_Rainfall**: Annual rainfall (in mm).
- **Fertilizer**: Amount of fertilizer used (in kg/ha).
- **Pesticide**: Amount of pesticide used (in kg/ha).
- **Yield**: Yield (Production/Area, in tonnes/ha).

This project's crop yield dataset includes comprehensive details on a range of factors affecting agricultural production in a variety of locations and seasons. A thorough explanation of the dataset, including its properties, rows, and columns, may be seen below.

Attribute	Type	Description	Example Values
Crop	String/Categorical	Type of crop cultivated	Wheat, Rice, Maize, Barley
Crop_Year	Integer	Year of cultivation	1997, 2000, 2005
Season	String/Categorical	Season during which the crop was cultivated	Kharif, Rabi, Summer, Winter
State	String/Categorical	State where the crop was cultivated	Maharashtra, Punjab, West Bengal
Area	Float	Area harvested (in hectares)	10.5, 20.3, 50.0
Production	Float	Production quantity (in tonnes)	100.5, 200.7, 500.0
Annual_Rainfall	Float	Annual rainfall (in mm)	700.5, 800.2, 900.0
Fertilizer	Float	Amount of fertilizer used (in kg/ha)	50.0, 100.0, 150.0
Pesticide	Float	Amount of pesticide used (in kg/ha)	10.0, 20.0, 30.0
Yield	Float	Yield (in tonnes/ha), calculated as Production/Area	10.0, 20.0, 25.0

Table1: Dataset Description

5. Methodology:

The source of my dataset is from Kaggle: https://www.kaggle.com/datasets/saincoder404/crop-yield-data-india

The section on methodology outlines the methodical approach and strategies employed to accomplish the project's goals. This covers the preprocessing procedures, the machine learning models that are employed, and the analysis and visualisation tools.

Instruments and Technology

PySpark: Used for feature engineering, machine learning model construction, and data preprocessing. Big data management is made scalable and effective with PySpark. Tableau: Used for exploratory data analysis and data visualisation. To better understand the data and the analysis's findings, Tableau facilitates the creation of engaging and intelligent visualisations.

Preparing Data

Managing Missing Values: To maintain data integrity, rows with missing values were located and eliminated. This stage is essential to guaranteeing the correctness of the analysis and preventing mistakes during model training.
Encoding Categorical Variables: StringIndexer was used to encode the categorical variables (Crop, Season, and State) by turning text labels into numerical indices. Then, oneHotEncoder was run over these indices to produce one-hot encoded vectors. These

vectors are better suited for machine learning models since they avoid any ordinal link between categories being assumed by the model.

Feature Engineering: All pertinent attributes (area, production, annual rainfall, fertiliser, pesticide, and encoded categorical variables) were combined into a features vector using VectorAssembler. The input for the machine learning models is this vector.
Data Splitting: Using randomSplit, the dataset was divided into training (80%) and testing (20%) sets. This guarantees that a subset of the data is used for training and unobserved data is used for evaluation, giving a realistic assessment of the models' performance.

Models for Machine Learning

The goal of linear regression is to forecast the yield using the input features.
The baseline model utilised to determine the association between the characteristics and the goal variable (Yield) was linear regression. It uses a linear combination of the input features to represent the target.

The purpose of the Decision Tree Regressor is to increase Yield's forecast accuracy.
To capture non-linear correlations between the characteristics and the target variable, a decision tree regressor was employed. Based on feature values, it divides the data into subsets and forecasts each subset's average value.
The aim of the Random Forest Regressor is to minimise overfitting and improve prediction accuracy.
The ensemble method known as the Random Forest Regressor was employed. By constructing several decision trees and averaging their forecasts, it lowers variance and strengthens the resilience of the model.

Classifying yields into binary groups (above or below the median yield) is done using logistic regression.
Decision Tree Classifier: A tree-based technique that divides the information into various groups.
The Random Forest Classifier is an ensemble technique that improves classification performance by combining several decision trees.

Visualization:

Tableau: To illustrate the data distribution, trends, and regression model outcomes, a number of plots and charts were made using Tableau.
Scatter plots for actual yields compared to predictions, bar charts for feature relevance, and line charts for trend analysis over time are a few examples.

6. Experimental Setup:

A detailed explanation of how to set up the environment required to run PySpark with Hadoop can be found in this section. Installing PySpark, Hadoop, and Homebrew on a macOS system is required for this configuration.

1.Setting up Homebrew
For macOS, Homebrew is a package manager that makes program installation easier.

Launch the terminal:
Use these instructions to install Homebrew:
Installing homebrew
Installing openjdk
Setting up Hadoop
Large datasets can be processed and stored in a distributed manner using the open-source Hadoop platform.
Use Homebrew to install Hadoop:
Installing Hadoop
Installing PySpark: PySpark is the Apache Spark Python API.
PySpark installation with Homebrew:
Installing and launching pyspark
Data Loading for the Experimental Protocol: To facilitate effective processing, the dataset was loaded into a PySpark DataFrame.
Data Cleaning: To guarantee data integrity, rows containing missing values were found and eliminated.
Encoding Categorical Variables: Crop, Season, and State are categorical text labels that were converted into numerical indices using StringIndexer.
These indices were subjected to OneHotEncoder, which produced one-hot encoded vectors that are more appropriate for machine learning techniques.
Feature engineering involves utilising VectorAssembler to combine numerical and encoded category attributes into a single features vector.

Data Splitting: To construct and assess the models, the dataset was divided into training (80%) and testing (20%) sets.

Model Development and Assessment:
three regression models (linear regression, decision tree regression, and random forest regression) were trained and assessed. The Random Forest Classifier, Decision Tree Classifier, and Logistic Regression model were the three classification models that were trained and assessed.
Metrics like R-squared (R2) and Root Mean Squared Error (RMSE), Accuracy, AUC were used to compare the models' performance.

The link between the characteristics and the goal variable (yield) was modelled using linear regression.

The decision tree regressor was employed to identify non-linear correlations between the target variable and the characteristics.

Random Forest Regressor: By lowering overfitting, the Random Forest Regressor ensemble approach enhanced prediction performance.

Visualization:
Tableau:
To visualise the data and the outcomes of the regression models, a number of plots were made. These included line charts for trend analysis over time, bar charts for feature importance, and scatter plots for actual yields compared to expected yields.

7. Result Discussion:

Model	RMSE	R-squared (R2)
Linear Regression	303.7	0.88
Decision Tree Regressor	139.56	0.97
Random Forest Regressor	203.1	0.94

Model	AUC	Accuracy
Logistic Regression	**0.84**	**0.84**
Decision Tree Classifier	**0.78**	**0.78**
Random Forest Classifier	**0.76**	**0.76**

Table2: Result Tables

8. Conclusion and Future Work:

Using a collection of agricultural variables, this project used Tableau for result visualisation and PySpark for data processing and machine learning in order to estimate crop yields. By using a methodical strategy that included data preparation, feature engineering, model training, and evaluation, the main goals were accomplished.

Important findings from the research consist of:

Model Performance: Measurable metrics like Root Mean Squared Error (RMSE) and R-squared (R^2) showed that the linear regression model performed reasonably well. These measurements gave rise to a quantitative understanding of the model's accuracy and the percentage of crop yield fluctuation that the model could account for. the agricultural yield dataset's performance indicators for several classification methods. The Random Forest Classifier, Decision Tree Classifier, and Logistic Regression models are among those that were assessed. Accuracy and AUC are the performance parameters utilised in the evaluation.

Understanding from Visualisations: Tableau visualisations such as scatter plots and residual plots aided in analysing the model's performance and pinpointing areas in need of development. The residual plot assisted in identifying any systematic errors, while the scatter plot showed how the projected and actual yields aligned.
Feature Importance: Important variables affecting agricultural productivity were discovered by feature engineering and model training. These insights, which emphasise the importance of specific variables like soil type, temperature, humidity, rainfall, fertiliser consumption, and pesticide usage, are helpful for agricultural planning and decision-making.

Techniques Effectiveness: PySpark and Tableau worked well together to manage and analyse big datasets. Tableau's powerful visualisation tools combined with PySpark's distributed data processing and machine learning capabilities allowed for a thorough examination of the dataset.
Potential future projects could include:
Although the project's main goals were met, there are still a number of areas that might use further development and investigation:
More Complex Modelling Methods:
Non-Linear Frameworks: To capture more intricate correlations between characteristics and crop productivity, investigate non-linear regression models or machine learning algorithms like decision trees, random forests, gradient boosting, and neural networks.
Use ensemble methods to aggregate the forecasts from several models, which may improve the predictive robustness and accuracy.
Other Sources of Data:
Weather Information: To increase model accuracy, incorporate more dynamic and detailed weather data, such as seasonal fluctuations, extreme weather occurrences, and climatic patterns.
Soil Health Metrics: To give a more thorough picture of how soil affects crop output, include soil health indicators like nutrient levels, pH, and organic matter content.
Enhancement and Augmentation of Data: Managing Missing Data Investigate sophisticated imputation methods to manage missing values more skilfully, guaranteeing the quality and completeness of your data.
Feature Engineering: To capture more complex relationships, keep improving feature engineering procedures, such as the development of interaction terms and polynomial features.
Analysis of Space and Time:
To comprehend regional differences in agricultural productivity and the impact of site-specific factors, perform spatial analysis.
Use temporal analysis to examine patterns throughout time in order to spot long-term trends and possible shifts in agricultural output.

Improve the interpretability of the model by employing strategies like SHAP (SHapley Additive exPlanations) values to give concise justifications for the predictions made by the model and the contributions made by the features.
Real-Time Predictions: Create systems for gathering and predicting data in real-time so that farmers and other stakeholders can act promptly and sensibly by making decisions based on forecasts and existing conditions.
Working with farmers, policymakers, and agricultural specialists, stakeholders can help test model predictions, get feedback, and improve models based on real-world experiences and field data.

Future research in these areas should improve crop output prediction modelling even more, offering more precise, dependable, and useful information for agricultural planning and decision-making. This project lays the groundwork for future research and development into the use of machine learning and data analytics in agriculture.

9. Social Impact:

There is a lot of potential for good social impact in the endeavour of predicting crop yields utilising modern data analytics and machine learning approaches. The following are some important areas in which the initiative can have a significant impact:

1. Food Security and Sustainability
2. Economic Benefits for Farmers
3. Policy and Decision Making

4. Environmental Impact
5. Social Equity
6. Educational and Research Contributions

This project's societal impact goes beyond the immediate gains from more accurate crop yield forecasts. It supports increased food security, environmental preservation, farmers' financial stability, and a more comprehensive understanding of sustainable agriculture. The initiative tackles significant issues in agriculture by utilising data analytics and machine learning, promoting a more robust and fair food system.

In addition to increasing production, the use of cutting-edge technologies in agriculture guarantees that the advantages of innovation are distributed to all facets of society, fostering a sustainable and inclusive future.

References:

1. Hussain, S. (2024). *Crop Yield Data India*. Kaggle.com. https://www.kaggle.com/datasets/saincoder404/crop-yield-data-india
2. Naveen (NNK). (2022, February 19). *How to Install PySpark on Mac (in 2022)*. Spark by {Examples}. https://sparkbyexamples.com/pyspark/how-to-install-pyspark-on-mac/
3. Code With Arjun. (2022, July 4). *INSTALL HADOOP ON MACOS (M1, M2) - Level Up Coding*. Medium; Level Up Coding. https://levelup.gitconnected.com/install-hadoop-on-macos-m1-m2-6f6a01820cc9
4. Tableau. (2018). *Tableau for Students*. Tableau Software. https://www.tableau.com/academic/students
5. Lipper, L., Thornton, P., Campbell, B. M., Baedeker, T., Braimoh, A., Bwalya, M., Caron, P., Cattaneo, A., Garrity, D., Henry, K., Hottle, R., Jackson, L., Jarvis, A., Kossam, F., Mann, W., McCarthy, N., Meybeck, A., Neufeldt, H., Remington, T., & Sen, P. T. (2014). Climate-smart agriculture for food security. *Nature Climate Change*, *4*(12), 1068–1072. https://doi.org/10.1038/nclimate2437
6. Bronson, K. (2018). Smart Farming: Including Rights Holders for Responsible Agricultural Innovation. *Technology Innovation Management Review*, *8*(2), 7–14. https://doi.org/10.22215/timreview/1135
7. Wolfert, S., Ge, L., Verdouw, C., & Bogaardt, M.-J. (2017). Big Data in Smart Farming – A review. *Agricultural Systems*, *153*, 69–80. https://doi.org/10.1016/j.agsy.2017.01.023
8. Wreford, A., Moran, D., & Adger, N. (n.d.). *Climate Change and Agriculture imPACtS, ADAPtAtiOn AnD mitigAtiOn*. Retrieved August 5, 2024, from https://www.fao.org/fileadmin/user_upload/rome2007/docs/Climate%20Change%20and%20Agr.pdf

Appendix1:

```
Last login: Mon Aug  5 04:14:41 on ttys000
sravanineelakantam@Sravanis-MacBook-Air ~ % /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
==> Checking for `sudo` access (which may request your password)...
Password:
==> This script will install:
/opt/homebrew/bin/brew
/opt/homebrew/share/doc/homebrew
/opt/homebrew/share/man/man1/brew.1
/opt/homebrew/share/zsh/site-functions/_brew
/opt/homebrew/etc/bash_completion.d/brew
/opt/homebrew

Press RETURN/ENTER to continue or any other key to abort:
==> /usr/bin/sudo /usr/sbin/chown -R sravanineelakantam:admin /opt/homebrew
==> Downloading and installing Homebrew...
Reset branch 'stable'
==> Updating Homebrew...
==> Installation successful!

==> Homebrew has enabled anonymous aggregate formulae and cask analytics.
Read the analytics documentation (and how to opt-out) here:
  https://docs.brew.sh/Analytics
No analytics data has been sent yet (nor will any be during this install run).

==> Homebrew is run entirely by unpaid volunteers. Please consider donating:
  https://github.com/Homebrew/brew#donations

==> Next steps:
- Run brew help to get started
- Further documentation:
    https://docs.brew.sh
```

Fig1: Installing brew

```
sravanineelakantam@Sravanis-MacBook-Air ~ % brew reinstall openjdk@11
==> Downloading https://ghcr.io/v2/homebrew/core/openjdk/11/manifests/11.0.24
Already downloaded: /Users/sravanineelakantam/Library/Caches/Homebrew/downloads/9074a5c218de33d0fa562968f9bb02048043b32a5e946dd693588c42886b0c3c--openjdk@11-11.0.24.bottle_manifest.json
==> Fetching openjdk@11
==> Downloading https://ghcr.io/v2/homebrew/core/openjdk/11/blobs/sha256:55565ebdad224446b1fc7dfb95ff248b474592e897e0f37fee42a926ba682853
Already downloaded: /Users/sravanineelakantam/Library/Caches/Homebrew/downloads/5952418512e7fccb919b6af67e13b67f62db8f781715f23751a1172aded7be9f--openjdk@11--11.0.24.arm64_sonoma.bottle.tar.gz
==> Reinstalling openjdk@11
==> Pouring openjdk@11--11.0.24.arm64_sonoma.bottle.tar.gz
🍺  /opt/homebrew/Cellar/openjdk@11/11.0.24: 669 files, 296.4MB
==> Running `brew cleanup openjdk@11`...
Disable this behaviour by setting HOMEBREW_NO_INSTALL_CLEANUP.
Hide these hints with HOMEBREW_NO_ENV_HINTS (see `man brew`).
sravanineelakantam@Sravanis-MacBook-Air ~ %
```

Fig2: Installing jdk

```
sravanineelakantam@Sravanis-MacBook-Air ~ % brew reinstall hadoop
==> Downloading https://ghcr.io/v2/homebrew/core/hadoop/manifests/3.4.0
Already downloaded: /Users/sravanineelakantam/Library/Caches/Homebrew/downloads/af84ce6b0126979cb4bdc8262dd817b491a14ee862659438e8d2f2cb31f3fd4a--hadoop-3.4.0.bottle_manifest.json
==> Fetching hadoop
==> Downloading https://ghcr.io/v2/homebrew/core/hadoop/blobs/sha256:3bc4d1af76f
Already downloaded: /Users/sravanineelakantam/Library/Caches/Homebrew/downloads/2e1cc93c46c4215f3ddef68cf039fa130a43933b37e33c7a7fab60327458282c--hadoop--3.4.0.arm64_sonoma.bottle.tar.gz
==> Reinstalling hadoop
==> Pouring hadoop--3.4.0.arm64_sonoma.bottle.tar.gz
🍺  /opt/homebrew/Cellar/hadoop/3.4.0: 20,170 files, 1.3GB
==> Running `brew cleanup hadoop`...
Disable this behaviour by setting HOMEBREW_NO_INSTALL_CLEANUP.
Hide these hints with HOMEBREW_NO_ENV_HINTS (see `man brew`).
```

Fig3: Instaliling Hadoop

```
sravanineelakantam@Sravanis-MacBook-Air ~ % echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> /Users/admin/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
zsh: no such file or directory: /Users/admin/.zprofile
sravanineelakantam@Sravanis-MacBook-Air ~ % brew install openjdk@11
Warning: openjdk@11 11.0.24 is already installed and up-to-date.
To reinstall 11.0.24, run:
  brew reinstall openjdk@11
sravanineelakantam@Sravanis-MacBook-Air ~ % brew install pyenv
Warning: pyenv 2.4.9 is already installed and up-to-date.
To reinstall 2.4.9, run:
  brew reinstall pyenv
sravanineelakantam@Sravanis-MacBook-Air ~ % pyenv install 3.11.5
pyenv: /Users/sravanineelakantam/.pyenv/versions/3.11.5 already exists
continue with installation? (y/N) y
python-build: use openssl@3 from homebrew
python-build: use readline from homebrew
Downloading Python-3.11.5.tar.xz...
-> https://www.python.org/ftp/python/3.11.5/Python-3.11.5.tar.xz
Installing Python-3.11.5...
python-build: use readline from homebrew
python-build: use zlib from xcode sdk

Installed Python-3.11.5 to /Users/sravanineelakantam/.pyenv/versions/3.11.5
sravanineelakantam@Sravanis-MacBook-Air ~ %
sravanineelakantam@Sravanis-MacBook-Air ~ % brew install pyenv-virtualenv
Warning: pyenv-virtualenv 1.2.4 is already installed and up-to-date.
To reinstall 1.2.4, run:
  brew reinstall pyenv-virtualenv
sravanineelakantam@Sravanis-MacBook-Air ~ % pyenv virtualenv 3.11.5 devenv
pyenv-virtualenv: `/Users/sravanineelakantam/.pyenv/versions/devenv' already exists.
sravanineelakantam@Sravanis-MacBook-Air ~ % pyenv shell devenv
pyenv: shell integration not enabled. Run `pyenv init' for instructions.
sravanineelakantam@Sravanis-MacBook-Air ~ % pyenv shell devenv
pyenv: shell integration not enabled. Run `pyenv init' for instructions.
sravanineelakantam@Sravanis-MacBook-Air ~ % pip install pyspark
zsh: command not found: pip
sravanineelakantam@Sravanis-MacBook-Air ~ % brew install apache-spark
==> Downloading https://formulae.brew.sh/api/formula.jws.json
######################################################################################### 100.0%
==> Downloading https://formulae.brew.sh/api/cask.jws.json
######################################################################################### 100.0%
Warning: apache-spark 3.5.1 is already installed and up-to-date.
To reinstall 3.5.1, run:
  brew reinstall apache-spark
sravanineelakantam@Sravanis-MacBook-Air ~ % pyspark
Python 3.12.4 (main, Jun  6 2024, 18:26:44) [Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
24/08/05 04:46:50 WARN Utils: Your hostname, Sravanis-MacBook-Air.local resolves to a loopback address: 127.0.0.1; using 172.16.10.248 instead (on interface en0)
24/08/05 04:46:50 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/08/05 04:46:50 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.5.1
      /_/

Using Python version 3.12.4 (main, Jun  6 2024 18:26:44)
Spark context Web UI available at http://172.16.10.248:4040
Spark context available as 'sc' (master = local[*], app id = local-1722829611110).
SparkSession available as 'spark'.
>>> 24/08/05 04:47:07 WARN GarbageCollectionMetrics: To enable non-built-in garbage collector(s) List(G1 Concurrent GC), users should configure it(them) to spark.eventLog.gcMetrics.youngGenerationGarbageColle
ctors or spark.eventLog.gcMetrics.oldGenerationGarbageCollectors
```

Fig4: Launching spark

```
>>> from pyspark.sql import SparkSession
>>> from pyspark.ml import Pipeline
>>> from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
>>> from pyspark.sql.functions import when, col
>>> from pyspark.ml.regression import LinearRegression, DecisionTreeRegressor, RandomForestRegressor
>>> from pyspark.ml.classification import LogisticRegression, DecisionTreeClassifier, RandomForestClassifier
>>> from pyspark.ml.evaluation import RegressionEvaluator, BinaryClassificationEvaluator
>>>
```

Fig5: Importing Libraries

```
>>> spark = SparkSession.builder.appName("Crop Yield Analysis").getOrCreate()
```

Fig6: Launching spark

```
>>> file_path = "/Users/sravanineelakantam/Downloads/crop_yield.csv"
>>> df = spark.read.csv(file_path, header=True, inferSchema=True)
```

Fig7: Loading the dataset

```
>>> df_clean = df.dropna()
>>> categorical_columns = ["Crop", "Season", "State"]
>>> indexers = [StringIndexer(inputCol=col, outputCol=col + "_index") for col in categorical_columns]
>>> encoders = [OneHotEncoder(inputCol=indexer.getOutputCol(), outputCol=indexer.getOutputCol() + "_vec") for indexer in indexers]
>>> stages = indexers + encoders
>>> feature_columns = [col for col in df_clean.columns if col not in ["Yield"] + categorical_columns]
>>> assembler_inputs = feature_columns + [encoder.getOutputCol() for encoder in encoders]
>>> assembler = VectorAssembler(inputCols=assembler_inputs, outputCol="features")
>>> stages.append(assembler)
>>> pipeline = Pipeline(stages=stages)
>>> pipeline_model = pipeline.fit(df_clean)
>>> df_transformed = pipeline_model.transform(df_clean)
>>> df_features_reg = df_transformed.select("features", "Yield")
```

Fig8: Data Processing

```
>>> median_yield = df_clean.approxQuantile("Yield", [0.5], 0.05)[0]
>>> df_with_target = df_clean.withColumn("Target", when(col("Yield") > median_yield, 1).otherwise(0)
>>> df_transformed_clf = pipeline_model.transform(df_with_target)
>>> df_features_clf = df_transformed_clf.select("features", "Target")
```

Fig9: binary classification target based on the median yield

```
>>> train_data_reg, test_data_reg = df_features_reg.randomSplit([0.8, 0.2], seed=1234)
>>> train_data_clf, test_data_clf = df_features_clf.randomSplit([0.8, 0.2], seed=1234)
```

Fig10: Split the dataset

```
>>> reg_models = {
...     "Linear Regression": LinearRegression(featuresCol="features", labelCol="Yield"),
...     "Decision Tree Regressor": DecisionTreeRegressor(featuresCol="features", labelCol="Yield"),
...     "Random Forest Regressor": RandomForestRegressor(featuresCol="features", labelCol="Yield")
... }
```

Fig11: Regression Models

```
>>> clf_models = {
...     "Logistic Regression": LogisticRegression(featuresCol="features", labelCol="Target"),
...     "Decision Tree Classifier": DecisionTreeClassifier(featuresCol="features", labelCol="Target"),
...     "Random Forest Classifier": RandomForestClassifier(featuresCol="features", labelCol="Target")
... }
```

Fig12: Classification Models

```
>>> for name, model in reg_models.items():
...     print(f"Training {name}...")
...     trained_model = model.fit(train_data_reg)
...     predictions = trained_model.transform(test_data_reg)
...     evaluator = RegressionEvaluator(labelCol="Yield", predictionCol="prediction", metricName="rmse")
...     rmse = evaluator.evaluate(predictions)
...     r2 = RegressionEvaluator(labelCol="Yield", predictionCol="prediction", metricName="r2").evaluate(predictions)
...     print(f"{name} - RMSE: {rmse}, R2: {r2}")
...
Training Linear Regression...
24/08/05 20:16:43 WARN Instrumentation: [86c35e4e] regParam is zero, which might cause numerical instability and overfitting
24/08/05 20:16:43 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
24/08/05 20:16:43 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.VectorBLAS
24/08/05 20:16:43 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.lapack.JNILAPACK
Linear Regression - RMSE: 303.70018875198963, R2: 0.8804556759443596
Training Decision Tree Regressor...
Decision Tree Regressor - RMSE: 139.5635918475589, R2: 0.9747545501899779
Training Random Forest Regressor...
Random Forest Regressor - RMSE: 203.13684309811413, R2: 0.9465169415206655
>>> for name, model in clf_models.items():
...     print(f"Training {name}...")
...     trained_model = model.fit(train_data_clf)
...     predictions = trained_model.transform(test_data_clf)
...     evaluator = BinaryClassificationEvaluator(labelCol="Target", rawPredictionCol="prediction", metricName="areaUnderROC
...     auc = evaluator.evaluate(predictions)
...     accuracy = predictions.filter(predictions["prediction"] == predictions["Target"]).count() / float(predictions.count(
...     print(f"{name} - AUC: {auc}, Accuracy: {accuracy}")
...
Training Logistic Regression...
Logistic Regression - AUC: 0.8434270755671809, Accuracy: 0.8433673469387755
Training Decision Tree Classifier...
Decision Tree Classifier - AUC: 0.7844525617370081, Accuracy: 0.7836734693877551
Training Random Forest Classifier...
Random Forest Classifier - AUC: 0.7662891362025233, Accuracy: 0.7660714285714286
```

Fig13: Evaluate Models

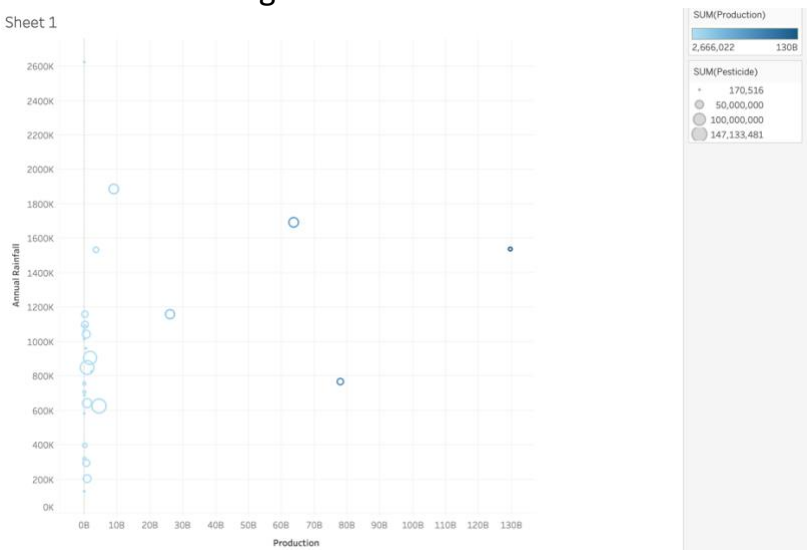

Fig14: Plot between Annual Rainfall and production

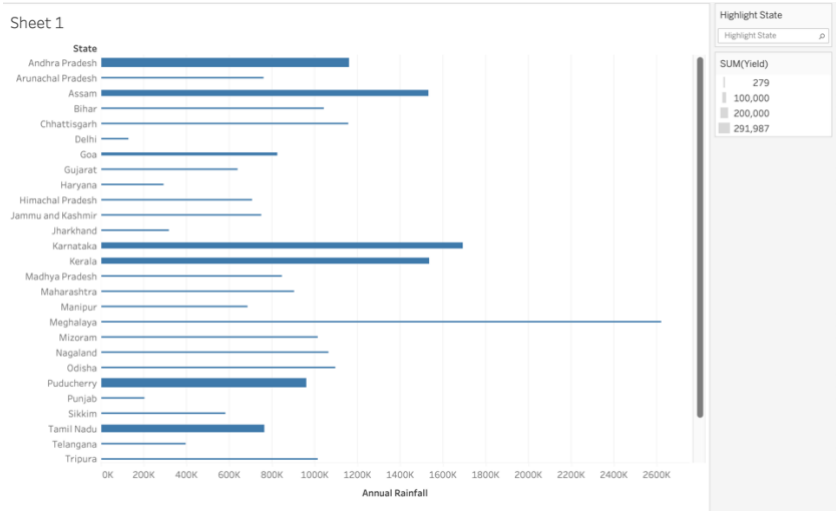

Fig15: plot between State and Annual Rainfall

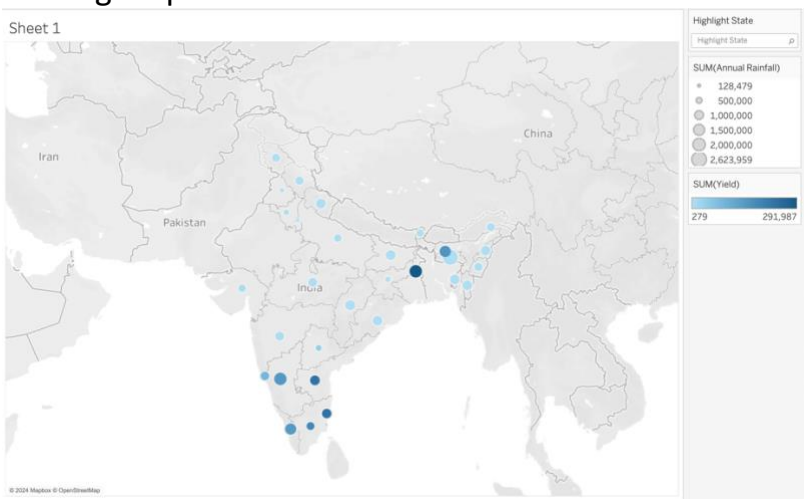

Fig16: Rainfall based on State

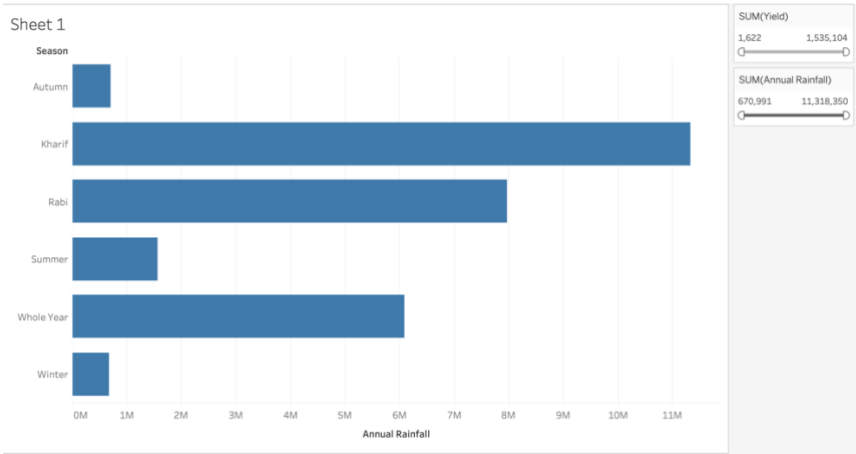

Fig17: plot between Season and rainfall

Sheet 1

State	Annual Rainfall	Area	Pesticide	Fertilizer	Production	Yield
Andhra Pradesh	1,160,140	238,925,335	62,107,243	31,258,901,414	26,076,218,605	229,735
Arunachal Pradesh	762,047	5,725,531	1,555,839	774,692,189	9,522,010	1,091
Assam	1,531,493	77,342,600	20,935,470	10,293,039,833	3,637,714,928	160,613
Bihar	1,042,239	167,995,431	45,310,803	22,202,119,051	544,953,534	3,115
Chhattisgarh	1,159,347	110,536,194	29,431,312	15,293,755,681	143,096,606	1,800
Delhi	128,479	797,903	210,849	104,531,936	2,666,022	2,663
Goa	824,746	2,749,585	750,611	358,107,780	2,193,998,349	87,276
Gujarat	641,180	222,255,493	60,067,112	29,930,280,999	807,581,678	5,472
Haryana	294,622	131,504,830	35,710,256	17,645,132,989	589,739,640	3,491
Himachal Pradesh	706,851	17,810,207	4,736,727	2,390,507,515	32,297,151	1,170
Jammu and Kashmir	752,008	18,743,187	5,141,242	2,534,720,338	30,298,377	825
Jharkhand	319,457	19,722,543	8,723,560	4,382,078,876	43,793,850	975
Karnataka	1,692,393	257,907,399	69,173,116	34,580,241,908	63,772,797,366	151,392
Kerala	1,536,641	31,853,731	9,647,595	4,744,603,224	129,700,649,853	147,710
Madhya Pradesh	847,829	487,969,617	135,990,782	67,088,002,957	834,490,323	2,922
Maharashtra	905,671	459,357,810	124,688,509	62,389,066,346	1,878,564,915	3,389
Manipur	686,844	5,727,639	1,556,678	800,982,110	18,764,635	2,316
Meghalaya	2,623,959	5,178,776	1,423,262	697,806,524	16,516,629	1,868
Mizoram	1,013,275	1,488,063	406,821	199,540,539	2,769,208	1,069
Nagaland	1,064,884	8,426,726	2,321,707	1,160,795,826	18,748,702	2,344
Odisha	1,098,412	124,646,660	33,385,548	16,638,622,133	194,080,325	2,387
Puducherry	959,916	653,541	170,516	84,807,289	493,815,573	232,164
Punjab	202,058	162,040,200	43,752,763	21,652,297,765	781,551,409	1,927
Sikkim	580,896	1,759,399	468,554	230,186,961	2,744,927	279
Tamil Nadu	765,414	110,372,797	29,563,050	14,732,390,935	78,051,759,253	185,813
Telangana	395,510	39,792,142	13,565,715	6,262,532,637	147,700,034	39,509
Tripura	1,015,520	6,278,528	1,712,264	856,174,567	16,952,054	1,891
Uttar Pradesh	625,576	542,672,635	147,133,481	72,452,818,301	4,442,585,302	4,169
Uttarakhand	1,087,097	22,471,891	6,007,196	3,126,578,846	179,697,348	2,854
West Bengal	1,883,456	237,682,262	66,126,649	29,704,801,336	8,941,179,120	291,987

Fig18: Text table

Appendix2:

Code:

```
from pyspark.sql import SparkSession
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder,VectorAssembler
from pyspark.sql.functions import when, col
from pyspark.ml.regression import LinearRegression, DecisionTreeRegressor,
RandomForestRegressor
from pyspark.ml.classification import LogisticRegression,
DecisionTreeClassifier, RandomForestClassifier
from pyspark.ml.evaluation import RegressionEvaluator,
BinaryClassificationEvaluator

# Initialize Spark session
spark = SparkSession.builder.appName("Crop Yield Analysis").getOrCreate()

# Load the dataset
file_path = "/mnt/data/crop_yield.csv"
df = spark.read.csv(file_path, header=True, inferSchema=True)

# Data Preprocessing
df_clean = df.dropna()
categorical_columns = ["Crop", "Season", "State"]
indexers = [StringIndexer(inputCol=col, outputCol=col + "_index") for col
in categorical_columns]
encoders = [OneHotEncoder(inputCol=indexer.getOutputCol(),
outputCol=indexer.getOutputCol() + "_vec") for indexer in indexers]
stages = indexers + encoders
feature_columns = [col for col in df_clean.columns if col not in ["Yield"]
+ categorical_columns]
assembler_inputs = feature_columns + [encoder.getOutputCol() for encoder in
encoders]
assembler = VectorAssembler(inputCols=assembler_inputs,
outputCol="features")
stages.append(assembler)
pipeline = Pipeline(stages=stages)
pipeline_model = pipeline.fit(df_clean)
df_transformed = pipeline_model.transform(df_clean)
df_features_reg = df_transformed.select("features", "Yield")

# Create a binary classification target based on the median yield
median_yield = df_clean.approxQuantile("Yield", [0.5], 0.05)[0]
df_with_target = df_clean.withColumn("Target", when(col("Yield") >
median_yield, 1).otherwise(0))

df_transformed_clf = pipeline_model.transform(df_with_target)
df_features_clf = df_transformed_clf.select("features", "Target")

# Split the data
train_data_reg, test_data_reg = df_features_reg.randomSplit([0.8, 0.2],
seed=1234)
train_data_clf, test_data_clf = df_features_clf.randomSplit([0.8, 0.2],
seed=1234)
# Regression Models
reg_models = {
```

```
    "Linear Regression": LinearRegression(featuresCol="features",
labelCol="Yield"),
    "Decision Tree Regressor":
DecisionTreeRegressor(featuresCol="features", labelCol="Yield"),
    "Random Forest Regressor":
RandomForestRegressor(featuresCol="features", labelCol="Yield")
}

# Classification Models
clf_models = {
    "Logistic Regression": LogisticRegression(featuresCol="features",
labelCol="Target"),
    "Decision Tree Classifier":
DecisionTreeClassifier(featuresCol="features", labelCol="Target"),
    "Random Forest Classifier":
RandomForestClassifier(featuresCol="features", labelCol="Target")
}

# Evaluate Regression Models
for name, model in reg_models.items():
    print(f"Training {name}...")
    trained_model = model.fit(train_data_reg)
    predictions = trained_model.transform(test_data_reg)
    evaluator = RegressionEvaluator(labelCol="Yield",
predictionCol="prediction", metricName="rmse")
    rmse = evaluator.evaluate(predictions)
    r2 = RegressionEvaluator(labelCol="Yield", predictionCol="prediction",
metricName="r2").evaluate(predictions)
    print(f"{name} - RMSE: {rmse}, R2: {r2}")

# Evaluate Classification Models
for name, model in clf_models.items():
    print(f"Training {name}...")
    trained_model = model.fit(train_data_clf)
    predictions = trained_model.transform(test_data_clf)
    evaluator = BinaryClassificationEvaluator(labelCol="Target",
rawPredictionCol="prediction", metricName="areaUnderROC")
    auc = evaluator.evaluate(predictions)
    accuracy = predictions.filter(predictions["prediction"] ==
predictions["Target"]).count() / float(predictions.count())
    print(f"{name} - AUC: {auc}, Accuracy: {accuracy}")

# Stop the Spark session
spark.stop()
```